1. Advanced Recon Strategy

Overview:

Elite hunting starts with discovering the infrastructure. Vertical recon (Subdomains) is common, but Horizontal recon (ASN/CIDR) is where the gems are found.

Command Line Workflow:

```
amass intel -org 'Target Name'
whois -h whois.radb.net -- '-i origin ASXXXXX' | grep -Eo '([0-9.]+){4}/[0-9]+'
subfinder -d target.com -all -recursive -o subs.txt
```

Hunter Strategy:

Identify the ASN of the target. Find all IP ranges (CIDR) owned by them. Scan these IPs for web services. You'll often find forgotten dev/staging environments that aren't behind the main WAF.

2. Systematic Fuzzing with Ffuf

Overview:

Directory discovery is useless without the right wordlists and filters. You must look for sensitive extensions and hidden API endpoints.

Command Line Workflow:

```
ffuf -w wordlist.txt -u https://target.com/FUZZ -mc 200,301,403 -e .php,.json,.xml,.bak
ffuf -w api_paths.txt -u https://api.target.com/v1/FUZZ -H 'Authorization: Bearer XXX'
```

Hunter Strategy:

Always filter out noise using -fc (filter code) or -fs (filter size). If the server returns a 403 on a directory, don't stop. Fuzz for files inside that directory. Developers often secure folders but forget the files inside.

3. Digging through Time: GAU & Wayback

Overview:

Finding endpoints that existed years ago but are still active is a goldmine for vulnerabilities like IDORs and Open Redirects.

Command Line Workflow:

```
gau target.com | grep -E '.php|.asp|.aspx|.jsp' | split-files
waybackurls target.com | gf redirect | anew redirects.txt
```

Hunter Strategy:

Use GAU to extract every URL ever indexed for the target. Sort them by extension. Look for parameters like ?url=, ?redirect=, or ?debug=true. These are prime targets for redirection or information leak bugs.

4. Low Hanging Fruit: Info Leaks

Overview:

Building confidence is key. Look for exposed .git folders, .env files, and juicy JavaScript comments.

Command Line Workflow:

```
git-dumper http://target.com/.git/ output_dir/
grep -rE 'api_key|secret|password' static/js/
trufflehog github --org=targetname
```

Hunter Strategy:

Information disclosure might not pay much ($100-$300), but they lead to Critical paths. A leak of an internal IP or a Slack webhook can be used for SSRF or internal phishing.

5. Real-time Subdomain Monitoring

Overview:

To get a 'First Blood', you must find the subdomain before anyone else. This requires an automated cron-job workflow.

Command Line Workflow:

```
while true; do subfinder -d target.com -silent | anew subs.txt | notify; sleep 3600; done
```

Hunter Strategy:

Set up a VPS. Run subdomain discovery every hour. Pipe the output to a tool like 'notify' (ProjectDiscovery) to alert your Slack/Discord. When a new staging sub appears, be the first to fuzz it.

6. Attacking the JavaScript

Overview:

Modern web apps are JS-heavy. Vulnerabilities are hidden within the client-side code.

Command Line Workflow:

```
subjs -i subs.txt | xargs -I %% linkfinder -i %% -o cli
secretfinder -i https://target.com/main.js -o cli
```

Hunter Strategy:

Extract all endpoints from JS files. Look for 'hidden' API routes like /api/v2/beta/admin. These are often unprotected because the developers think no one can see them without the documentation.

7. Beyond Web: Port Scanning

Overview:

Bug hunting isn't just about Port 80/443. Other services offer direct RCE paths.

Command Line Workflow:

```
naabu -list subs.txt -p 80,443,8080,8443,9000,3000 -silent
nmap -sV -p 9000 <target_ip> --script=http-title
```

Hunter Strategy:

Look for development ports (3000, 8080). Often, Docker containers or Jenkins instances are left exposed. Finding an exposed 'Spring Boot Actuator' on port 8080 is an instant P2/P1 bug.

8. Hidden Parameter Mining

Overview:

Vulnerabilities like XSS and SQLi are often in parameters not visible in the UI.

Command Line Workflow:

```
arjun -u https://target.com/login.php -w params.txt
x8 -u https://target.com/search -w wordlist.txt --get
```

Hunter Strategy:

Use Arjun to find hidden GET/POST parameters. If you find a parameter name like 'debug' or 'test_admin', try to inject a payload. These 'Developer-only' parameters often skip security checks.

9. Writing for the Impact

Overview:

A bug is only as good as the report. If the triager can't see the impact, you won't get a bounty.

Command Line Workflow:

```
TEMPLATE:
Title: Full Account Takeover via IDOR on /api/users/update
Impact: I can modify the email of any user by changing the ID.
CVSS: 9.1 (Critical)
```

Hunter Strategy:

Always include a Video PoC if possible. Clearly explain the 'Business Impact'. Instead of saying 'There is an IDOR', say 'I can access the credit card data of all 50,000 customers'.

10. Final Execution Checklist

Overview:

Before you submit, ensure your sanity. Verification is the difference between a Pro and a Script Kiddie.

Command Line Workflow:

```
1. Is it in scope? (Check wildcards)
2. Can you reproduce it in a clean browser?
3. Is it a duplicate? (Check public disclosure)
```

Hunter Strategy:

Don't rush to report. Spend 30 minutes trying to 'Escalate' the bug. Can your XSS steal a session cookie? Can your IDOR reach an Admin account? Impact = $$$.